Tumbler: A Layered Survival Firewall for the AMD ROCm Stack under Production AI Workloads

Chun-Hung Wang*, Clement Lin, Jeremy Liao
AMD
Chun-Hung.Wang@amd.com, Clement.Lin@amd.com, Jeremy.Liao@amd.com
* Corresponding author

ABSTRACT

Production AI serving stacks share AMD GPUs across many concurrent tenants, run for hours or days continuously, and treat transient hardware and driver faults as a routine operational cost. The default ROCm runtime, designed for HPC single-tenant correctness, calls std::abort() on a VM fault, a wedged SDMA ring, or a parked HSA signal — turning a localized fault into a process-wide outage that takes every co-tenant down with it. We argue that this is the right behaviour for the workload ROCm was originally designed for, and the wrong behaviour for production AI. We present Tumbler, an opt-in survival firewall layered across the three lines of the ROCm stack — the CLR (HIP host runtime), the ROCr (HSA runtime), and the amdgpu kernel driver — that bounds the wait time at every known abort or unbounded-park site and returns a graceful failure to the calling request instead of aborting the process. Tumbler defaults to off (byte-identical to stock) so it is safe to land upstream. We have submitted Tumbler as five independent pull requests against ROCm/rocm-systems and ROCm/amdgpu, and validated the integrated stack on a multi-tenant MI300X production serving workload that previously wedged within single-digit minutes; with Tumbler enabled, the same workload remained stable for over 11 hours.

Keywords: AI serving, ROCm, AMD GPU, fault tolerance, runtime survivability, HIP, HSA, amdgpu kernel driver, MI300X.

1 Introduction

Picture yourself driving on the highway. The vehicle infotainment touchscreen — rendered by AMD amdgpu — hits a transient GPU fault mid-frame. What you want from that system is unambiguous: drop the bad frame, log the fault, and keep the dashboard, the windshield HUD, and the surround-view cameras alive. What you do not want is for the GPU driver to crash the entire in-vehicle infotainment process and freeze every safety-critical display you depend on. The argument is intuitive when the failure surface is a windshield. It is the same argument when the failure surface is an AI inference fleet serving thousands of concurrent tenants on a node of MI300X GPUs.

Figure 1. The vehicle cockpit argument. The central infotainment touchscreen (amdgpu) has hit a GPU fault, but the dashboard cluster and windshield HUD continue rendering normally. Drop the bad pipeline, keep the safety surfaces alive.

Modern AI serving stresses the AMD GPU runtime in ways it was not originally designed for: a single inference process hosts hundreds of concurrent tenant requests sharing the same GPUs, SDMA engines, HSA signals, and KFD pinned-memory windows. Long-running tensor ops, multi-tenant sharing, RDMA-Write pipelines, and eviction during inference all produce transient HW/driver faults whose blast radius in stock ROCm is the whole process — every co-tenant inside dies even though only one request was affected. This is a workload-shape mismatch rather than a bug: ROCm was built for HPC single-tenant correctness, where a VM fault means data corruption and aborting is exactly the right response. Production AI inverts the assumption — the cost of killing the process is several orders of magnitude larger than the cost of dropping one request — and needs an opt-in escape hatch that does not change default behaviour for legitimate single-tenant HPC users.

We present Tumbler, a layered opt-in survival firewall for the ROCm stack. Tumbler installs bounded-wait knobs at every known abort site and every known unbounded-park site across the three layers that matter — the CLR host runtime, the ROCr HSA runtime, and the amdgpu kernel driver — and converts those sites from "abort the process" or "block the caller forever" into "fail this one call gracefully and let the higher-level scheduler decide what to do". Every knob defaults to off, so a system that does not

set any Tumbler knob behaves byte-identically to stock ROCm.

Contributions:

- We characterize the workload-shape mismatch between HPC single-tenant correctness and production AI serving, and identify the abort/park sites it exposes across the ROCm stack.
- We design Tumbler, a layered opt-in survival firewall spanning CLR, ROCr, amdkcl, and amdgpu, comprising 10 self-contained environment-variable and modparam knobs.
- We measure the firewall on a real multi-tenant MI300X serving workload and show that a stack that previously wedged in single-digit minutes can be made stable for over 11 hours with Tumbler knobs enabled.
- We submit the entire design upstream as five independent pull requests against ROCm/rocm-systems and ROCm/amdgpu, and publish an integrated meta-repository so the combination can be rebuilt deterministically.

2 Background: The ROCm Stack Today

AMD ROCm is the open-source GPU compute stack for AMD Instinct and Radeon hardware. From the application down to the device, three layers matter for survivability analysis: CLR (the HIP host runtime, a.k.a. rocclr) manages streams and waits on HSA signals; ROCr (the HSA runtime) owns queues, signals, SDMA copies, and the user-mode side of fault delivery; and amdgpu (the upstream Linux kernel driver) manages VRAM, GTT, and KFD, with an amdkcl helper layer providing DKMS shims for non-mainline kernel APIs. A request issued by an AI serving framework travels down through these three layers into the GPU and back. At every layer there is a wait point where the caller can be parked indefinitely if the lower layer fails to make progress, and historically an abort site that escalates a fault into a process-wide termination.

3 Why ROCm Has No Built-in Survival Firewall

The HSA specification, the ancestor of ROCr, was designed with the assumption that one logical user process owns the GPU. Under that assumption, a hardware VM fault, a queue exception, or a memory corruption signal from the device is by definition unrecoverable — the calling process has by hypothesis already lost data integrity, and the only safe action is to abort.

ROCr implements this faithfully. The three abort sites in the runtime — the memory critical-error handler, the HW exception handler, and the VM fault handler — all call std::abort() unconditionally. CLR's WaitForSignal() polling loop has a 4-second per-call timeout in its call to Hsa::signal_wait_scacquire(), but the outer while loop has no wall-clock cap, so a permanently wedged signal parks the caller thread forever. SDMA writers spin on os::YieldThread() until they can publish an address. The amdgpu kernel driver waits on dma_fence_wait_any_timeout() with MAX_SCHEDULE_TIMEOUT inside kcl_drm_suballoc_new() and on a global mutex inside the GTT window path. Every one of these is correct behaviour when one process owns one GPU.

There is one existing knob that approaches the survivability problem from a different angle: HSA_SIGNAL_WAIT_ABORT_TIMEOUT. When set, the HSA runtime aborts the process when a signal wait exceeds the configured deadline. This is the right tool to backstop runaway kernels in a batch HPC job. It is the wrong tool for AI serving: it still aborts the process, just on a timer instead of on a fault. A serving stack that aborts at the deadline loses every co-tenant request in flight — even though those requests are still healthy.

4 Failure Modes Under AI Serving Load

We observed five distinct failure modes during multi-tenant production serving on customer MI300X nodes: SDMA ring stall (writers spin in AcquireWriteAddress() on os::YieldThread() indefinitely); per-request VM fault (ROCr's handler calls std::abort() and every co-tenant dies with it); HSA signal wait parked (CLR's WaitForSignal() outer loop has no wall-clock cap); GTT window starvation (every VRAM-touching ioctl parks on the global mman.gtt_window_lock mutex held by one in-flight SDMA copy); and KFD orphan pin (a pinned BO's owning process exits without unpinning, holding VRAM open and starving subsequent allocations). Stack signatures and root causes vary, but the pattern is uniform: one request hits an unrecoverable condition, the stack escalates it to a process-wide abort or an indefinite park, and every co-tenant request is collateral damage.

5 Tumbler Design — A Layered Survival Firewall

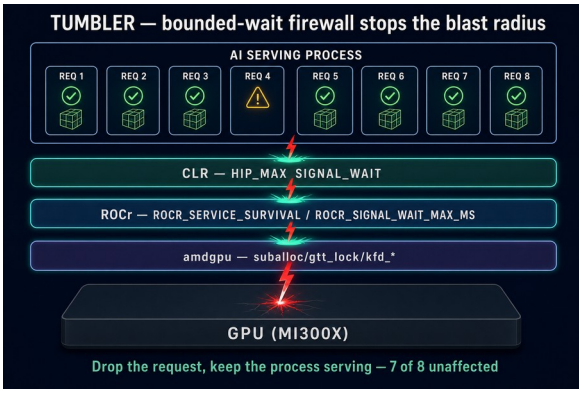

Figure 2. Tumbler installs a bounded-wait firewall at each layer of the stack. The fault is stopped at the layer closest to it, the affected request fails gracefully, and unrelated co-tenant requests continue serving.

Tumbler treats each ROCm layer as a firewall plane. At every plane we install (a) a bounded-wait conversion of the corresponding unbounded-park site, and (b) where the

plane is an abort site, an opt-in non-abort branch that surfaces the failure as a return code instead of as a process-wide std::abort(). Two design rules are fixed across the whole stack: the default is off (byte-identical to stock), and the failure mode at the deadline is graceful — the calling code receives a well-defined error, never an abort.

The protections decompose cleanly along the three boundary planes of the runtime — CLR, ROCr, and amdgpu — each catching a specific class of failure as it tries to propagate up the stack. Table 1 summarises what each layer guards against and how it does so. We leave individual knob names and tuning details to the upstream pull requests cited in Section 7 and to the meta-repository, so that this paper focuses on the protection model rather than on configuration mechanics.

Layer	Guards against	How it protects
CLR (HIP host runtime)	A host thread parked forever on a wedged HSA completion signal; the serving scheduler cannot reroute or fail the affected request.	Bounded outer-loop wait on WaitForSignal(). At the deadline the signal is force-completed and the caller returns, letting the upper layer decide whether to retry or fail the one request.
ROCr (HSA runtime)	std::abort() at the three runtime.cpp abort sites (memory error, non-ECC HW exception, VM fault) — one fault takes down the whole process and every co-tenant request with it. Plus SDMA writers spinning forever in AcquireWriteAddress(), and InterruptSignal waits parked.	Opt-in non-abort branches at every abort site (ECC stays abort-fatal for data integrity). Bounded caps on the signal-wait inner loop and on the SDMA yield loop. Failure surfaces to the caller as an HSA_STATUS error, never as a process abort.
amdgpu (Linux kernel driver)	A wedged SDMA ring holding the global GTT-window mutex and starving every VRAM-touching ioctl on the device. Pinned BOs whose owner exited without unpinning, permanently holding VRAM. Unbounded dma_fence_wait inside suballoc.	Bounded timeouts at every fence and lock site (suballoc, GTT window, KFD free / unpin). KFD pin-reaper background subsystem harvests orphan pins on a deadline. Failure surfaces to user space as -ETIME, never as a hung ioctl.

Table 1. Tumbler's three layers of defense.

6 Evaluation

We validated the integrated Tumbler stack on a customer MI300X serving box running a representative production AI inference workload. The baseline (stock ROCm) wedged within single-digit minutes in our reproducer; with Tumbler knobs enabled at the gold values, the same workload ran continuously for over 11 hours.

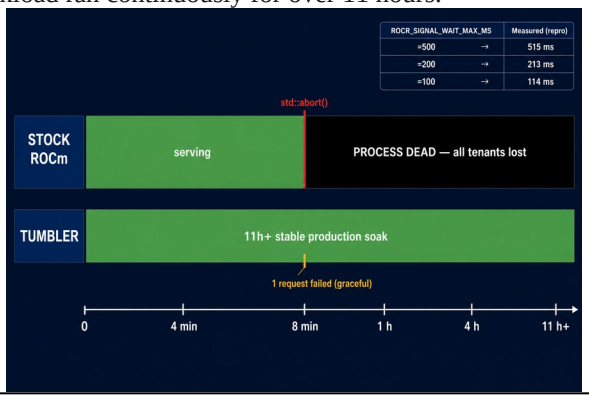

Figure 3. Before-and-after timeline. Stock ROCm: process abort within minutes. Tumbler: 11+ hours of stable serving on the same workload.

We further isolated the ROCr non-abort signal-wait cap (ROCR_SIGNAL_WAIT_MAX_MS) under a microbenchmark that holds the signal wedged. The measured time-to-return scales linearly with the configured cap (Table 2), confirming that the deadline mechanism is sharp and predictable.

ROCR_SIGNAL_WAIT_MAX_MS	Measured wall-clock to return
500	515 ms
200	213 ms
100	114 ms

Table 2. Sub-second linear scaling of the ROCr signal-wait cap.

Beyond raw stability, the firewall isolates faults per-request rather than per-process: a fault hitting one tenant's request on one GPU fails that single request gracefully, while every co-tenant on the same MI300X node continues serving uninterrupted.

7 Upstream Status

The entire Tumbler design has been submitted upstream as five independent pull requests so that each protection plane can be reviewed and merged on its own merit: ROCm/rocm-systems #6328 (CLR) and #6329 (ROCr), and ROCm/amdgpu #214 (amdkcl), #215 (amdgpu GTT), and #216 (amdgpu KFD pin reaper). No PR depends on any other; each can be enabled in isolation.

The integrated combination of all five is published as the meta-repository github.com/AFDEAPAC/Tumbler with two submodules pinned at integration tips (chun-wan/rocm-systems @ tumbler-integrated for the CLR + ROCr changes and chun-wan/amdgpu @ tumbler-integrated for the three amdgpu changes), so the exact stack measured in Section 6 can be rebuilt deterministically.

8 Related Work

HSA_SIGNAL_WAIT_ABORT_TIMEOUT is the closest existing upstream knob; it bounds the wait but escalates at the deadline to a process-wide abort, which is the behaviour Tumbler deliberately avoids for AI serving. The two are complementary: set ABORT_TIMEOUT strictly larger than ROCR_SIGNAL_WAIT_MAX_MS to keep a hard backstop on top of the graceful cap. Windows GPU TDR resets the GPU on long-running kernels but is opaque to the user-mode runtime and offers no per-request isolation inside a single host process. Linux IOMMU recovery isolates faulting devices via page-fault routing one layer below the GPU runtime and does not address user-mode abort sites in CLR or ROCr. NVIDIA MPS partitions a single GPU across multiple processes for inter-process isolation; Tumbler is intra-process isolation for the case where dozens of tenants share one Python serving process by design.

9 Conclusion and Future Work

Tumbler is small. The aggregate diff is on the order of a few hundred lines of added code spread across three repos, every knob defaults to off, and every knob is independently reviewable. The argument is not that the implementation is novel — the argument is that the underlying assumption (one host process, one GPU, fault = data corruption) is no longer the workload that matters most, and that the runtime needs a small, opt-in set of escape hatches to be usable as the substrate for production AI.

Future work centres on integrating Tumbler with the AI workload itself rather than leaving knob values as static environment variables: auto-tuning each deadline from observed workload signatures (tail-latency budget, batch dwell time, in-flight count), reinforcement-learning-driven survival policies, and feedback channels that surface graceful-fail events back into the inference framework so the higher-level scheduler can adjust load.

References

[1] ROCm/rocm-systems PR #6328 — rocclr: optional bounded HSA signal wait via HIP_MAX_SIGNAL_WAIT. https://github.com/ROCm/rocm-systems/pull/6328

[2] ROCm/rocm-systems PR #6329 — rocr: non-abort service-survival envs. https://github.com/ROCm/rocm-systems/pull/6329

[3] ROCm/amdgpu PR #214 — drm/amdkcl: bounded kcl_drm_suballoc_new wait via suballoc_timeout_ms. https://github.com/ROCm/amdgpu/pull/214

[4] ROCm/amdgpu PR #215 — drm/amdgpu: bounded mman.gtt_window_lock acquisition via gtt_lock_timeout_ms. https://github.com/ROCm/amdgpu/pull/215

[5] ROCm/amdgpu PR #216 — drm/amdgpu: KFD RDMA-pin bounded-wait + orphan reaper subsystem. https://github.com/ROCm/amdgpu/pull/216

[6] AFDEAPAC/Tumbler — integrated meta-repository. https://github.com/AFDEAPAC/Tumbler

[7] ROCm contributing guidelines. https://github.com/ROCm/ROCm/blob/develop/CONTRIBUTING.md#pull-requests

[8] HSA Foundation. HSA Runtime Programmer's Reference Manual.